# ShipFast — Provision Infrastructure with Terraform

Week 4, Session 2 assignment · Capstone Milestone 3 · Mohammed Awwad · progress report, 28 October 2026

| REPOSITORY | BUCKET | REGION | ACCOUNT | TERRAFORM | PROVIDER | FIRST APPLY |
|---|---|---|---|---|---|---|
| github.com/ MohammedAwwad2/ shipfast-devops | created and live (see Part D) | us-east-1 | IAM user, not root | v1.17.0 on linux_amd64 (WSL2) | hashicorp/aws 5.100.0 | 28 Oct 2026, 01:22 |

This report covers the part of Milestone 3 that is finished: Terraform installed, the `terraform/` folder created and protected, `main.tf` written against the starter template, and a first `init` and `apply` that created a real bucket in the account. Tasks 5 and 6 — reading the state, the four experiments and `NOTES.md` — are not done yet and are listed at the end as what remains.

## Part A — Installing Terraform

Terraform was installed inside WSL2 (Ubuntu) by following the official HashiCorp documentation rather than by downloading a zip by hand: add the HashiCorp GPG key to `/usr/share/keyrings/`, register the apt repository signed by that key, then `apt install terraform`. That way `apt upgrade` keeps it current and the binary's provenance is verifiable.

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor \
     -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
     https://apt.releases.hashicorp.com $(lsb_release -cs) main" \
     | sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt update && sudo apt install terraform -y
```

Both files landed at 00:58 on 28 October, and the package manager now owns the binary. The pre-flight checks from the Week 4 progress check pass:

```
$ cat /etc/apt/sources.list.d/hashicorp.list
deb [arch=amd64 signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
    https://apt.releases.hashicorp.com resolute main

$ dpkg -l terraform
ii  terraform   1.17.0-1   amd64   Terraform enables you to safely and predictably create, change ...

$ terraform version
Terraform v1.17.0
on linux_amd64

$ aws sts get-caller-identity
{
    "UserId": "AIDAXA74...",
    "Account": "4831*******",
    "Arn": "arn:aws:iam::4831*******:user/adminUser"
}
```

Terraform 1.17.0 is above the 1.6.0 floor the assignment requires, and the caller is an IAM user, not the root account. Evidence: `docs/evidence/m3/00-install-and-identity.txt`.

## Part B — The folder, and keeping state out of Git

The `terraform/` folder was created inside the capstone repo at 01:01. The `.gitignore` already written in Week 3 covers Terraform: the provider cache and every state file are ignored, while the provider lock file is not. That split is deliberate — the lock file belongs in Git because it pins the provider version for everyone on the team, and the state file must not, because it is a plaintext dump of every resource attribute.

```
# .gitignore (the Terraform part)
.terraform/
*.tfstate*

$ git ls-files | grep -E "tfstate|\.terraform/"      # prints nothing
$ git ls-files terraform/
terraform/.terraform.lock.hcl
terraform/main.tf
```

## Part C — Writing main.tf

All nine TODOs in the starter template were filled in. Three of them carried the decisions worth writing down:

### The provider carries only a region.

No access keys, anywhere. Terraform reads the same credential chain the AWS CLI does, so the `aws configure` profile from A1 is already enough. A key hard-coded here would end up in Git and in state.

### The bucket name is computed, never typed.

S3 bucket names are global across every AWS account, so a literal name would collide with a stranger's bucket. A `data "aws_caller_identity"` lookup asks AWS who the caller is, and the account ID in the name makes it unique without hard-coding anything.

### Versioning and encryption are separate resources.

In AWS provider 5.x these are no longer inline blocks on the bucket; they are their own resources that point back at it by `aws_s3_bucket..id`. That reference is also what gives Terraform the ordering — the bucket has to exist before anything can be configured on it.

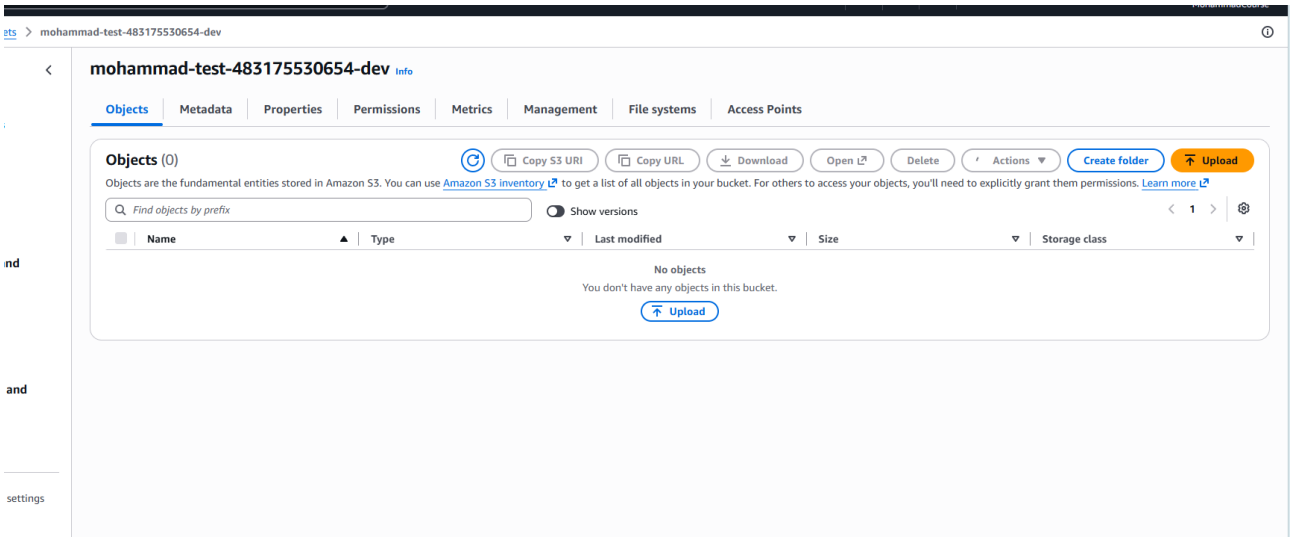
## Part D — init and apply

`terraform init` ran at 01:18. It downloaded hashicorp/aws 5.100.0 into `.terraform/` and wrote `.terraform.lock.hcl` a few seconds later. `terraform apply` followed at 01:22 and created all three resources — the bucket, its versioning configuration and its default encryption. The state file records the apply, and the bucket is live in the account.

```
$ cd terraform
$ terraform init      # Installing hashicorp/aws v5.100.0 ... Terraform has been successfully initialized!
$ terraform apply     # 3 added, 0 changed, 0 destroyed
```

Committed as `feat: add terraform infrastructure for shipfast`.

### The bucket in the S3 console




S3 console, Objects tab, straight after the first apply — the bucket exists and is empty, which is what the assignment expects at this stage. Blur the account ID before sharing this outside the repo.

## Part E — What is left

The remaining work, in the order the assignment lists it:

| Task | What is missing | What to do |
|---|---|---|
| **Task 2** | Bucket name and tags do not yet match the brief | The name must be shipfast-artifacts-<environment>-<account id>, and the tags must be Environment, ManagedBy = terraform and Project = shipfast-devops. Changing the name forces a replacement, so this is a destroy-and-create on the next apply. |
| **Task 3** | fmt, validate and a written plan prediction | Run fmt and validate, and write the prediction in NOTES.md before running plan: how many resources, and which values read "known after apply". |
| **Task 4** | Verify from the AWS CLI, not from Terraform | aws s3 ls, get-bucket-versioning, get-bucket-encryption, then terraform output. Save the output. |
| **Task 5** | Read the state | terraform state list, state show on one address, head -40 terraform.tfstate. Find serial, lineage and the real bucket ID, and write one sentence on what links an address to an ID. |
| **Task 6** | The four experiments | Re-apply unchanged; add a tag by hand in the console then plan and apply; plan -var environment=staging; plan with the state file moved aside. Predict each one first, then write what happened and why. |
| **Evidence** | docs/evidence/m3/ and the five questions | Save the apply output and the three CLI checks, keep the console screenshot, and answer the five questions in terraform/NOTES.md. |

Note on safety: the bucket must be left standing once the evidence is saved — Session 3 begins by destroying it, after the code has been rewritten as a module. And `-auto-approve` stays out of every apply run on a laptop.